

Modules 19–20: Fine-Tuning & Model Customization › Agentic AI

PROGRAM	ASSESSMENT CODE	TOTAL TIME	STACK
Data Science	M20-A1	4 Hours	Python 3

GENERAL INSTRUCTIONS

Attempt all sections. Write all code in Python 3 unless stated otherwise. For coding tasks, submit your .py source file along with a screenshot of the program output in your terminal or IDE. Label every task clearly with its section and task number. All work must be your own — submissions that appear AI-generated without the Section D disclosure process will not be evaluated.

A

Section A — Concept Application

Answer all questions. Each question presents a real-world scenario followed by a concept-specific question. Questions 1–3 draw from Module 19 (Fine-Tuning & Model Customization); Questions 4–6 draw from Module 20 (Agentic AI).

SCENARIO

S
1

You are building a chatbot for a food delivery platform that must recommend dishes to users based on their dietary restrictions, past order history, and real-time menu availability from partnered restaurants.

Question: Your team is debating whether to use prompt engineering, RAG, or fine-tuning to power this chatbot. The menu data updates daily and the user base is diverse. Evaluate all three approaches against this use-case and justify which combination — or single approach — you would recommend, and why the others fall short for this specific problem.

SCENARIO

S
2

You are working on an AI team at a food delivery company and have been asked to fine-tune a large language model to understand food-specific customer service queries. Your infrastructure has a single consumer-grade GPU with 8 GB VRAM, and full fine-tuning of even a 7B-parameter model is not feasible.

Question: Explain what LoRA (Low-Rank Adaptation) modifies in a pre-trained model compared to full fine-tuning. Why does this make it viable under severe VRAM constraints? What trade-off, if any, does the team accept by using LoRA instead of full fine-tuning?

SCENARIO

S
3

You are debugging a fine-tuned complaint classifier deployed by a food delivery company. The model was trained on 6,000 labelled complaint tickets and performs well on common issues — wrong item delivered, cold food — but fails on rare complaints like 'delivery partner was rude during a religious event' that were absent from training data.

Question: Identify two fundamental limitations of fine-tuning that this scenario exposes. For each limitation, explain whether re-fine-tuning with more data, switching to RAG, or using few-shot prompting in the system prompt would be the more appropriate remedy — and state the condition under which your chosen remedy is valid.

SCENARIO

S
4

You are designing an AI agent for a food delivery app. The agent can check restaurant availability, estimate delivery time, and place orders on a user's behalf. A customer sends the message: 'Order 2 portions of Masala Dosa from Saravana Bhavan for delivery at 7 PM tonight.'

Question: Trace how this agent processes the request using the Perceive › Think › Act loop. For each step, specify exactly what the agent perceives from the input, what reasoning it applies in the Think step (including any ambiguities it must resolve), and which tools it calls in the Act step and in what order. What happens if Saravana Bhavan is unavailable at 7 PM — how does the loop continue?

SCENARIO

S
5

Your food delivery platform's support and analytics teams currently ask a developer to write one-off SQL queries whenever they need order, restaurant, or delivery-partner data — a slow process that requires database access for every analyst. You are asked to design a Database MCP Server so that any MCP-compatible AI assistant can query this data directly.

Question: Which tools would you expose (e.g. `list_tables`, `describe_table`, `run_select_query`) and would you grant read-only or read-write access — and why? Name one specific question a support agent would realistically ask (e.g. about a delayed order), and explain what safeguard stops the connected LLM from returning sensitive data, such as a customer's phone number or saved payment details, that the query didn't need.

SCENARIO

S
6

You are optimising the customer support agent for a food delivery platform using the ReAct (Reason + Action) pattern. A user sends the query: 'My order #FD4521 was 40 minutes late and the food arrived cold. I want either a full refund or a significant discount on my next order.'

Question: Walk through at least two complete Reason › Action › Observation cycles that this ReAct agent would execute to handle this query. For each cycle, state what the agent reasons, what action it takes (tool called), and what it observes from the result. Explain why separating the reasoning step from the action step is critical when the agent is making customer-facing decisions that have financial consequences.

B

Section B — Practical Coding Tasks

Complete all tasks. Tasks 1–2 anchor on Module 19 and Module 20 individually; Tasks 3–4 combine both modules. The application domain for all tasks is a food delivery platform. Follow the requirements listed under each task.

Task 1: AI Technique Decision Engine

Build a Python function that recommends the most suitable AI technique — Prompt Engineering, RAG, or Fine-Tuning — for a given food delivery use-case based on a structured set of input criteria. (Foundational — single concept, direct rule-based implementation.)

- Define a function `recommend_technique(update_frequency, data_in_external_docs, needs_custom_behaviour, latency_budget)` that accepts four string arguments ('high'/'low', 'yes'/'no', 'yes'/'no', 'low'/'high').
- Implement a decision tree inside the function that maps the four inputs to one of the three techniques, with at least five distinct rule paths.
- Return a dict with keys 'technique' and 'justification', where 'justification' is a 2-sentence explanation of why the technique was selected.
- Call the function with at least four distinct food-delivery scenarios — for example: real-time menu recommendations, complaint categorisation, order confirmation message generation, and domain-specific FAQ answering — and print the recommendation for each.

Task 2: Tool-Calling Delivery Agent

Build a Python class `FoodDeliveryAgent` that simulates a tool-calling agent capable of routing customer queries to the correct internal tool and executing it. (Applied — tool-call routing combined with class-based state management.)

- Define at least four tool functions outside the class: `check_restaurant_status(name)`, `get_estimated_delivery_time(order_id)`, `apply_discount(order_id, reason)`, and `file_complaint(order_id, issue)`. Each must return a formatted string response. Alongside each function, define an accompanying MCP-style schema — a dict with keys `name`, `description`, and `parameters` — and print all four schemas once before running any test queries, matching the tool schema format covered in Module 20's MCP sessions.
- The `FoodDeliveryAgent` class must have a `think(query)` method that parses keywords in the query string and maps them to the correct tool function.
- The `think()` method must also update an internal log list with each `(query, tool_called, result)` tuple so that the session history is tracked.
- Test the agent with at least five varied customer queries covering all four tools, and print both the tool output and the session log at the end.

Task 3: Memory-Aware Strategy Selector

Build a Python agent that maintains per-user memory across interactions and uses it to decide whether to respond using a prompt template, a simulated RAG lookup, or a simulated fine-tuned model call. (Intermediate — module-19 technique selection logic combined with module-20 agent memory.)

- Define three strategy functions: `prompt_template_response(query)`, `rag_lookup_response(query)`, and `fine_tuned_response(query)`. Each must return a simulated but contextually relevant string response for a food delivery domain.
- Maintain a memory dict keyed by `user_id` storing fields: `interaction_count`, `preferred_cuisine`, `last_complaint` (initially `None`). Update this dict after every interaction.
- Implement a `decide_strategy(user_id, query)` function that selects fine-tuned for complaint-type queries after 3+ interactions, `rag_lookup` for menu or item queries, and `prompt_template` for all other queries — and returns both the strategy name and the response.
- Simulate a sequence of at least five interactions for the same `user_id`, showing how the strategy selection evolves as memory updates, and print the full memory state after each interaction.

Task 4: Multi-Agent Order Lifecycle System

Build a console-based multi-agent system for a food delivery platform where three specialised agents collaborate through a coordinator to handle the complete lifecycle of a customer order. (Integrative — multi-step agent orchestration combining module-19 technique decisions with module-20 multi-agent architecture.)

- Create three agent classes — `OrderAgent`, `DispatchAgent`, and `SupportAgent` — each with a `single_process(data)` method. `OrderAgent` creates an order summary dict from raw user input; `DispatchAgent` assigns a delivery partner and ETA from a simulated partners list; `SupportAgent` classifies post-delivery feedback as 'positive', 'complaint', or 'escalate' and selects a response technique (prompt, RAG, or fine-tuned) based on the classification.

- Create a Coordinator class with a `run_lifecycle(customer_input, feedback)` method that passes outputs between agents in the correct order — OrderAgent › DispatchAgent › SupportAgent — collecting each agent's output.
- After the full lifecycle completes, the Coordinator must print a structured lifecycle report showing: order summary, dispatch details, feedback classification, technique selected, and total simulated processing time.
- Run the full lifecycle for at least two different customer scenarios (e.g. a smooth delivery and a complaint scenario) and demonstrate that the SupportAgent selects different techniques for each.

C

Section C — Mini Capstone Project

Mini Project: Agentic Food Delivery Assistant

Objective:

Build a console-based multi-tool AI agent for a food delivery platform that integrates fine-tuning technique selection, tool calling, per-session memory, and multi-step ReAct-style reasoning into a single interactive program — combining the key concepts from Section B into a working capstone.

Your project must:

- The program must present a menu-driven interface with at least four options: (1) Place Order, (2) Track Order, (3) File Complaint, (4) Get Personalised Recommendations — plus an exit option that prints the session summary.
- Each menu option must invoke a dedicated tool function (`place_order()`, `track_order()`, `file_complaint()`, `get_recommendations()`), and the agent must log every tool call with its arguments and result in a `session_log` list. Also define an MCP-style schema (name, description, parameters) for each tool function and print the full schema set once at program start, before the menu loop begins.
- The agent must maintain a `session_memory` dict that stores: `orders_placed`, `complaints_filed`, `preferred_cuisine`, and `interaction_count` — updating it after every menu action so that later interactions are informed by earlier ones.
- A `decide_response_strategy(action_type, memory)` function must be called for every action and must return one of three strategies — 'prompt', 'rag', or 'fine_tuned' — based on the action type and the current memory state. The selected strategy must be printed alongside the tool result.
- On exit, the program must print a full session report listing: all actions taken in order, the tool invoked and strategy selected for each, and a summary of the final `session_memory` state.

D

Section D — AI-Augmented Learning

STEP 1 • BUILD WITH AI

Use an AI tool of your choice (ChatGPT, Claude, GitHub Copilot, etc.) to help you write a Python program that:

- Implements a simplified ReAct (Reason + Action) loop for a food delivery customer support agent that processes a user complaint query.
- Executes at least two complete Reason › Action › Observation cycles before producing a final response — and prints each cycle's thought, action call, and observation clearly to the console.
- Includes at least two callable tool functions (e.g. `lookup_order_status(order_id)` and `check_refund_eligibility(order_id)`) that the agent invokes during the action step.
- Handles the case where a tool returns an error or 'not found' result and incorporates that observation into the next reasoning step rather than crashing.

STEP 2 • TEST & DEBUG (WITHOUT AI)

Then, working without AI, test the code and find at least one bug, limitation, or improvement in the AI's solution. Fix it yourself.

SUBMIT

All 3 items required

- 1 The exact prompt(s) you gave the AI tool.
- 2 The AI's original code and your corrected version.
- 3 A 3–4 line note explaining what you changed and why the AI's version needed it.